SWE-EVO: Benchmarking Coding Agents in Long-Horizon Software Evolution Scenarios

Tue Le[1,*] **Minh Vu Thai Pham**[1,*] **Dung Nguyen Manh**[2] **Huy Nhat Phan**[1] **Nghi D. Q. Bui**[3,†]
[1]FPT Software AI Center, Ha Noi, Viet Nam
[2]School of Computing and Information Systems, University of Melbourne
[3]Center of AI Research, VinUniversity, Ha Noi, Viet Nam

Existing benchmarks for AI coding agents focus on isolated, single-issue tasks such as fixing a bug or adding a small feature. However, real-world software engineering is a long-horizon endeavor: developers interpret high-level requirements, coordinate changes across many files, and evolve codebases over multiple iterations while preserving functionality. We introduce SWE-EVO, a benchmark for this long-horizon software evolution challenge. Constructed from release transitions in seven mature open-source Python projects, SWE-EVO comprises 48 release-sized tasks requiring multi-step modifications spanning an average of 21 files, validated against test suites averaging 874 tests per instance. Experiments reveal a striking capability gap: the best model reaches only 25% on SWE-EVO, while `gpt-5.2` drops from 72.80% on SWE-Bench Verified to 22.92% on SWE-EVO, showing that current agents struggle with sustained, multi-file reasoning. We also propose *Fix Rate*, a metric capturing partial progress on these complex, long-horizon tasks.

`https://github.com/SWE-EVO/SWE-EVO`

1. Introduction

Large language models (LLMs) have achieved remarkable progress in automating software engineering (SE) tasks, including code generation (Bui et al., 2023; Chen et al., 2021a; Li et al., 2022; Manh et al., 2023; To et al., 2023; Wang et al., 2023; Wei et al., 2023; Zhuo et al., 2024), bug fixing (Jimenez et al., 2023; Xia et al., 2024), and test synthesis (Chen et al., 2022; Jain et al., 2024; Wang et al., 2024b). These advancements have facilitated the emergence of AI-powered coding agents capable of assisting or automating key aspects of the software development lifecycle (Fan et al., 2023; Gao et al., 2025; He et al., 2025; Zhang et al., 2023).

Building on these capabilities, multi-agent systems have rapidly emerged to address long-horizon challenges in software engineering. By coordinating specialized agents for navigation, localization, patching, and verification, these frameworks improve scalability and performance over single-agent architectures. Recent systems (Nguyen et al., 2025b; Phan et al., 2024; Wang et al., 2024d; Yang et al., 2024a) illustrate this shift in real-world repositories. Industry adoption reflects this momentum: over 90% of engineering teams now integrate generative AI into SE workflows, up from 61% in 2024 (DORA Research Program, 2025).

To evaluate these agents rigorously, benchmarks have become essential. Early efforts like HumanEval (Chen et al., 2021b) focused on function-level code completion (Chen et al., 2021a),

[*]Equal contribution [†]Project lead
Correspondence to: Tue Le <tueldt1@fpt.com>, Minh Vu Thai Pham <minhpvt@fpt.com>,
Dung Nguyen Manh <manhdung.nguyen.1@student.unimelb.edu.au>, Huy Nhat Phan <huypn168@gmail.com>,
Nghi D. Q. Bui <bdqnghi@gmail.com>.

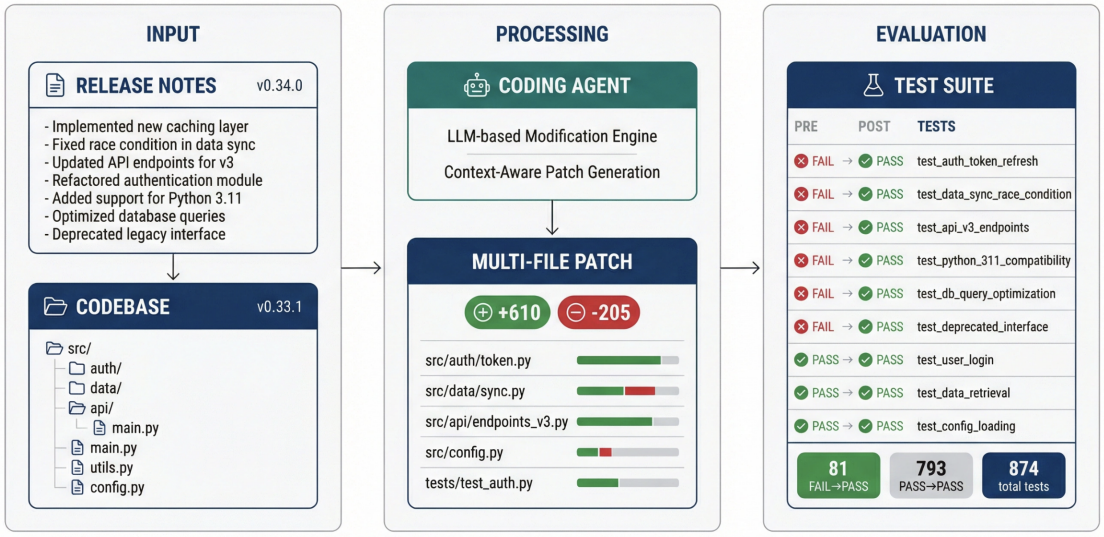


Figure 1 | Overview of the SWE-EVO task pipeline. **Input**: the agent receives release notes specifying multiple changes (bug fixes, new features, maintenance) alongside the full codebase at the start version. **Processing**: a coding agent generates a multi-file patch spanning dozens of files. **Evaluation**: the patch is validated against a comprehensive test suite comprising FAIL→PASS tests (verifying the required changes) and PASS→PASS tests (ensuring no regressions).

while SWE-Bench (Jimenez et al., 2023) curated real-world GitHub issues requiring verifiable patches for isolated problems (Jimenez et al., 2023). SWE-Bench is now a de facto standard for assessing multi-agent capabilities in practical coding scenarios.

While recent models reach around 75% on SWE-Bench-Verified (e.g., `GPT-5.2` (OpenAI, 2025c)) and around 40% on the full leaderboard (e.g., OpenCSG Starship at 39.67% (Jimenez et al., 2024)), the benchmark shows signs of saturation on isolated tasks. More importantly, SWE-Bench focuses on discrete issue resolution and does not capture the core SE challenge: continuous evolution of existing systems (Kaur and Singh, 2015; Singh et al., 2019). In practice, up to 80% of effort is spent maintaining and evolving legacy code, requiring coordinated changes across modules, versions, and specifications (Kaur and Singh, 2015; Singh et al., 2019).

This gap between benchmark tasks and real-world evolution scenarios motivates our central research question: *Given an existing codebase, can multi-agent LLM systems autonomously evolve the system in response to dynamic input requirements, demonstrating sustained planning, adaptability, and innovation across long-horizon tasks?*

To address this gap, we introduce SWE-EVO, a benchmark for autonomous software evolution rather than single-issue repair. SWE-EVO uses release notes, commit histories, and versioned snapshots from mature open-source Python projects to construct tasks where agents interpret Software Requirement Specifications (SRS), plan multi-step modifications, and evolve codebases across releases. SWE-EVO comprises 48 release-sized tasks across 7 repositories (Table 1 and Figure 3). We describe construction in Section 3 and results in Section 4.

Figure 1 illustrates the SWE-EVO pipeline: from release notes and a start-version codebase, the agent generates a multi-file patch validated by comprehensive tests. Software evolution (Figure 7 in Appendix B) requires SRS alignment, holistic program understanding, and coordinated changes, unlike SWE-Bench's isolated issue setting (Figure 2).

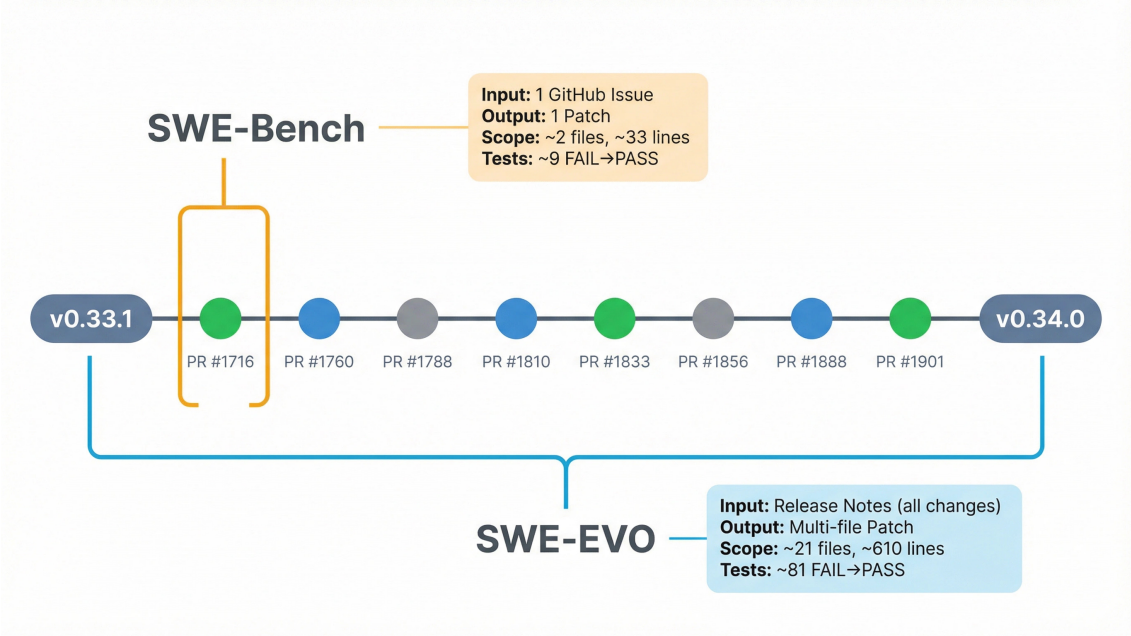


Figure 2 | Comparison of SWE-Bench and SWE-EVO. SWE-Bench tasks agents with resolving a single GitHub issue to produce one patch. SWE-EVO requires agents to interpret release notes and implement comprehensive changes that resolve multiple PRs, develop new features, and fix multiple bugs to evolve the codebase to a new version.

Our evaluation with OpenHands, SWE-agent, and 18 models reveals a significant capability gap: the best model (`gpt-5.4`) resolves only 25% of SWE-EVO, while `gpt-5.2` drops from 72.80% on SWE-Bench Verified to 22.92%. Larger models generally outperform smaller variants. Failure analysis suggests that stronger models more often misinterpret nuanced release notes, while weaker models struggle with tool use and syntax errors.

2. Related Work

2.1. Code Generation and SE Benchmarks

Code generation evaluation has moved from function-level tasks (Austin et al., 2021; Chen et al., 2021a; Hendrycks et al., 2021) to repository-level SE challenges. Early benchmarks such as HumanEval, MBPP, and APPS remain largely single-file and increasingly saturated (Zhou et al., 2023), while newer efforts such as CodeMMLU (Nguyen et al., 2025a) and CodeWiki (Hoang et al., 2025) broaden evaluation to code understanding and documentation. SWE-bench (Jimenez et al., 2024) shifted the field to verifiable patches for real GitHub issues, with curated subsets (OpenAI, 2024b) and extensions across languages (Zan et al., 2025), multimodal inputs (Yang et al., 2024b), live collection (Badertdinov et al., 2025; Zhang et al., 2025b), and enterprise-scale settings (Scale AI, 2025). EnConda-Bench (Kuang et al., 2025) isolates environment-configuration trajectories, including setup planning, diagnosis, repair, and execution. SWE-EVO instead targets version-to-version software evolution, where agents must implement release-level changes in the codebase rather than fix injected configuration faults. Prior benchmarks therefore emphasize isolated issue resolution or narrow subskills more than multi-step evolution across commits and longer planning horizons.

2.2. Software Engineering Agents

Interactive bug-fixing work (Chen et al., 2023; Xia and Zhang, 2023, 2024; Yuan et al., 2024) and autonomous systems such as Devin AI (Cognition, 2024) established the basis for SE agents. SWE-agent (Yang et al., 2024a) emphasized the agent–computer interface; OpenHands (Wang et al., 2024d) presented a multi-agent platform evaluated on 15 benchmarks; AutoCodeRover (Zhang et al., 2024) paired LLMs with AST-based search; and Agentless (Xia et al., 2024) showed that a simple localization-repair pipeline can remain competitive. Multi-agent designs further explore role specialization and coordination, including AgentCoder (Huang et al., 2023), AgileCoder (Nguyen et al., 2025b), and CodeAct (Wang et al., 2024c). Model-side progress includes post-training on real SE data (Carbonneaux et al., 2025; Chen et al., 2025; DeepSeek AI, 2025; Kimi Team, 2025b; Luo et al., 2025; Wei et al., 2025) and synthetic-data generation (Pham et al., 2025). A related line studies self-evolving agents and tool construction (Cai et al., 2024; Qian et al., 2024; Qiu et al., 2025; Robeyns et al., 2025; Wang et al., 2024a, 2025, 2024e; Zhang et al., 2025a) (Appendix C).

2.3. Context Engineering for Long-Horizon Agents

Long-horizon agents depend on effective context management, which is central to SWE-EVO because tasks unfold across large codebases and many turns. Surveys (Hua et al., 2025; Mei et al., 2025) frame the area around retrieval, processing, and management, tracing a path from prompt engineering and RAG to context engineering. Representative methods include compression (Ge et al., 2024), retrieval schemes such as Self-RAG (Asai et al., 2024), RAPTOR (Sarthi et al., 2024), and GraphRAG (Gutierrez et al., 2024), and memory systems such as MemGPT (Packer et al., 2023) and hierarchical storage (Zhong et al., 2024) that extend beyond single sessions. Meta Context Engineering (Ye et al., 2026) pushes this further by optimizing context assembly, reporting 89.1% on SWE-bench Verified versus 70.7% for hand-engineered baselines. These ideas are directly relevant to SWE-EVO, where agents must reason over long specifications and patches spanning many files.

3. SWE-EVO Dataset

SWE-EVO is a benchmark constructed from release notes and version histories of popular open-source repositories, capturing real software evolution scenarios where coding agents must interpret high-level software requirement specifications, plan and implement multi-step modifications across versions, and ensure that the evolved system passes validation tests aligned with those specifications.

3.1. Benchmark Construction

The construction of SWE-EVO consists of three major phases: (1) repository selection and data scraping, (2) candidate selection and filtering, and (3) execution-based filtering. We describe each phase in turn.

Stage I: Repository Selection and Data Scraping. To maximize reproducibility and comparability, we inherit repositories and execution environments from SWE-bench and it keeps our benchmark "plug-and-play" for existing SWE agents. Concretely, we begin by collecting samples from SWE-bench (Jimenez et al., 2024) and from SWE-gym (Pan et al., 2024) as our seed pool of task instances. Both resources already give us: a real repository snapshot, an executable environment, and a set of tests tied to a human-authored change.

Stage II: Candidate Selection and Filtering. Unlike SWE-Bench, which frames tasks as resolving a single issue, we target evolving a codebase between two release versions. We therefore create

		Mean	Max
Issue Text	Length (Words)	2390.5	22344
Codebase	# Files (non-test)	363	1046
	# Lines (non-test)	78K	272K
Gold Patch	# Lines edited	610.5	4113
	# Files edited	20.9	105
	# Func. edited	51.0	379
Tests	# Fail to Pass	81.4	2774
	# Total	874.0	8552

Table 1 | Average and maximum numbers characterizing different attributes of a SWE-EVO task instance. Statistics are micro-averages calculated without grouping by repository.

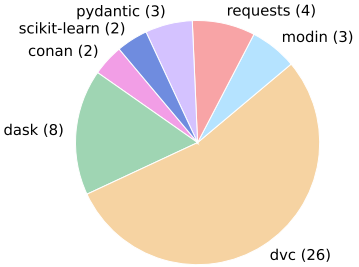


Figure 3 | Distribution of SWE-EVO tasks (in parenthesis) across 7 open source GitHub repositories that each contains the source code for a popular, widely downloaded PyPI package.

candidate instances by selecting only those samples whose base commit corresponds exactly to a version tag of the repository (i.e., a release snapshot). For each such candidate, we define the problem statement as the release-note delta between that version and its next tagged version; the agent's job is to implement the specified changes across the codebase. This framing treats a release as the unit of evolution, matching how users and maintainers typically receive a mixture of fixes, features, and maintenance changes in open-source projects. Importantly, SWE-EVO is not intended to imitate a single clean pull request or one atomic commit. Instead, it evaluates whether an agent can coordinate the full release-level behavioral delta. This formulation intentionally withholds an oracle decomposition into a sequence of separately specified PRs: agents may still work iteratively during their trajectory, but evaluation is based on the final evolved repository state. A sequential PR-by-PR variant is a useful future setting, but it would provide cleaner intermediate supervision and move closer to repeated single-issue repair.

Stage III: Execution-Based Filtering. Following SWE-Bench, we validate each candidate by applying the instance's test patch content and logging test outcomes *before* and *after* the remaining patch content is applied. We retain only instances that exhibit at least one `FAIL_TO_PASS` test (i.e., a test that fails pre-patch and passes post-patch), ensuring a measurable behavioral change attributable to the required evolution. We additionally discard candidates that trigger installation or runtime errors under the benchmark environment. The resulting set comprises evolution tasks with verifiable behavioral deltas and stable execution characteristics.

After the automatic filtering steps yielded a few hundred candidate release transitions, we used human effort to manually verify task quality and scale the benchmark down to 48 high-confidence instances. This quality-first pass checks alignment among the release note, linked PR/issue context, gold patch, and executable tests. Figure 3 reports their distribution across repositories, while Table 1 summarizes key characteristics. Compared with SWE-Bench, SWE-EVO has longer specifications, broader patches, and heavier test suites (Figure 4), making each human-curated release transition substantially larger in engineering scope than a single-issue repair task. We therefore treat benchmark scale along two axes: number of instances and scope per instance. SWE-EVO is smaller on the former, but much larger on the latter, so we avoid over-interpreting close leaderboard gaps while using these release-sized tasks as a stress test of software-evolution capability.

Figure 4 makes this scale difference explicit. Across specification length, edited lines, edited files, edited functions, `FAIL_TO_PASS` tests, and total tests, SWE-EVO is consistently larger than SWE-

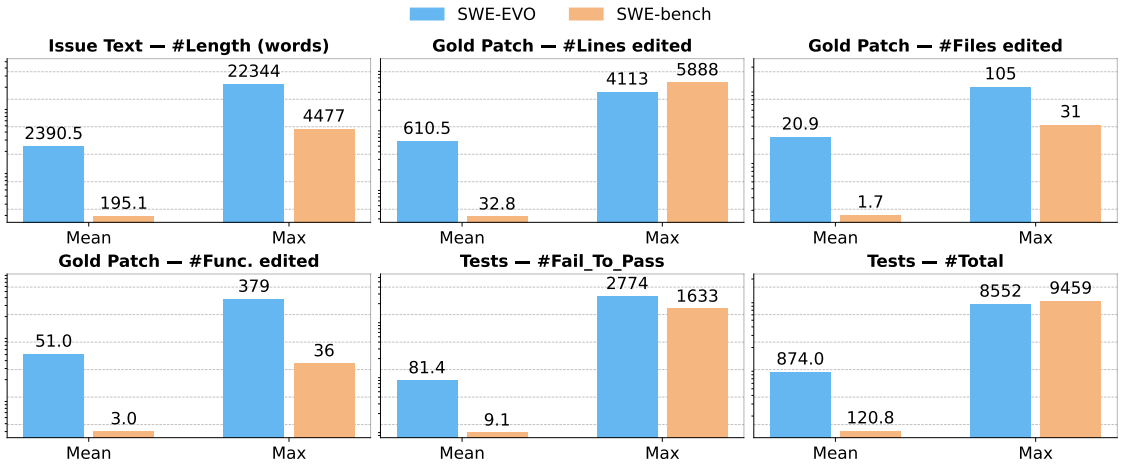


Figure 4 | SWE-EVO is markedly more demanding than SWE-Bench, with longer specifications, broader patch scope, and heavier test suites in both mean and max.

Bench in both average and worst-case size. These differences explain why SWE-EVO stresses release-level software evolution: agents must interpret broader requirements, coordinate changes across more code, and satisfy substantially heavier verification suites.

3.2. Task Formulation

Having described how SWE-EVO instances are constructed, we now formalize the task definition and evaluation metrics.

Model input. A model is provided with (i) a release-note-centered specification of the intended change (bug fix or feature refinement), consisting of the release note itself and, in our default setting, the text of any PRs or issues explicitly linked from that release note (Appendix N.1), and (ii) a complete codebase at the *pre-release* commit. The model must produce edits to the codebase that implement the described change. In practice, we represent a solution as a patch file that specifies which lines to modify across files. For clarity, we refer to the patch generated by the model as the `model patch`, while the ground-truth patch extracted from the start-version to end-version change inside the instance is referred to as the `gold patch`.

Evaluation Metrics. Consistent with the evaluation setup of SWE-bench (Jimenez et al., 2024), we use the **Resolved Rate (%)** as the primary metric, representing the proportion of task instances successfully solved by the agent. In addition, we report the **Patch Apply Rate (%)**, which measures the percentage of generated patches that are syntactically valid and can be applied to the codebase without errors. Beyond the hard-score **Resolved Rate (%)**, we introduce a soft-score, **Fix Rate (%)**, to provide a more fine-grained assessment of SWE-EVO while remaining consistent with Resolved Rate.

Resolved Rate. Following SWE-Bench, this metric focuses on two categories in the test outcomes:

- `FAIL_TO_PASS` tests that were initially failing (`FAILED` or `ERROR`) but pass (`PASSED` after applying the gold patch.

- `PASS_TO_PASS` tests pass both before and after the gold patch and serve as regression checks to ensure unrelated functionality is preserved.

The **Resolved Rate** of an instance is equal 1 if *all* tests in both `FAIL_TO_PASS` and `PASS_TO_PASS` are `PASSED`; otherwise, it is 0. Hence, this metric imposes a strict binary criterion: an instance is counted as resolved only if every relevant test succeeds.

Fix Rate: A Soft Metric. While Resolved Rate provides a clear pass/fail signal, it may hide meaningful partial progress, especially when instances contain thousands of tests (Table 1, Figure 4). In such cases, models may fix many failing tests without fully resolving the instance. To capture this progress, we introduce **Fix Rate**, a soft metric that measures the fraction of `FAIL_TO_PASS` tests successfully fixed.

Fix Rate also enforces a regression constraint: if any `PASS_TO_PASS` test fails after applying the patch, the instance receives a score of 0. This rewards partial fixes while penalizing regressions. We use this constraint because a valid evolution should implement the requested change without breaking existing behavior. Let F_i denote the `FAIL_TO_PASS` tests and P_i the `PASS_TO_PASS` tests for instance i. The **Fix Rate** is defined in Equation 1 as:

$$\text{Fix Rate}(i) = \begin{cases} \frac{\#\{t \in F_i \mid t \text{ passes}\}}{|F_i|}, & \text{if all } t \in P_i \text{ pass;} \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

The overall **Fix Rate (%)** across all N instances is $100 \times \frac{1}{N} \sum_{i=1}^{N} \text{Fix Rate}(i)$.

Fix Rate lies in $[0, 1]$ and is consistent with Resolved Rate: an instance is resolved when its Fix Rate equals 1. Aggregated across instances, this metric captures partial progress that binary Resolved Rate cannot reflect, while remaining aligned with it (Table 6). We keep strict Fix Rate as the main soft metric because valid software evolution should make progress without regressing existing behavior. For diagnostics, the same test logs can also report relaxed `FAIL_TO_PASS` progress and `PASS_TO_PASS` preservation separately, distinguishing incomplete implementations from regressions (Appendix J). Fix Rate is therefore an execution-based progress signal, not a complete measure of code quality, maintainability, or patch minimality.

3.3. Features of SWE-EVO

SWE-EVO differs from SWE-Bench along several key dimensions. First, it presents substantially richer supervision: longer specifications, broader patches (more lines, files, and functions edited), and heavier test suites (Figure 4). Second, unlike SWE-Bench where each task maps to a single pull request, SWE-EVO instances may aggregate multiple PRs, yielding a wide range of difficulty levels (see Appendix F). Finally, evaluation is robust: at least one `FAIL_TO_PASS` test validates each gold patch (81% have two or more), with an additional mean of 793 `PASS_TO_PASS` regression tests per instance.

4. Experiments

4.1. Experiment Setup

We evaluate SWE-EVO with two coding-agent scaffolds and a diverse model suite covering frontier proprietary systems and open-weight models.

Agents and Model Selection. We evaluate SWE-EVO using two established coding agent frameworks: **OpenHands** (Wang et al., 2024d) (with CodeActAgent, max 100 iterations) and **SWE-agent** (Yang et al., 2024a) (max 100 LLM calls). We test 18 state-of-the-art LLMs spanning five providers: OpenAI (10 models including GPT-5 series, o3, GPT-4.1, GPT-4o, and GPT-oss-120b), DeepSeek (3 models), Zhipu AI (3 models), Qwen (1 model), and Moonshot AI (2 models). The full list of models with references is provided in Table 3 in the Appendix.

For all reasoning models, we use a medium reasoning effort setting, which balances inference cost and accuracy.

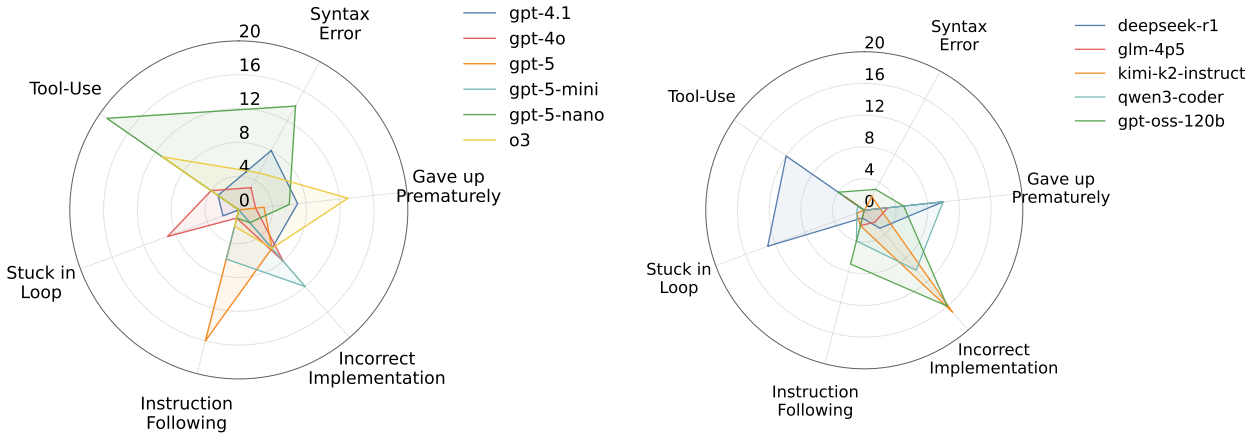
4.2. Performance on SWE-EVO

Each SWE-EVO task is specified by a release-note entry, optionally augmented with linked PR/issue text. To prevent models from retrieving future code or online PR content during evaluation, we block internet access and provide the linked PR/issue text directly in our default *release-note + PR/issue context* setting (Appendix N.1). Table 2 reports results for OpenHands and SWE-agent, alongside each model's SWE-bench Verified score when available.

	Model	SWE-EVO (OpenHands)		SWE-EVO (SWE-agent)		SWE-Bench Verified (bash only)
		% Resolved	% Apply	% Resolved	% Apply	% Resolved †
OpenAI	gpt-5.4	**25.00**	97.92	**25.00**	97.92	—
	gpt-5.2	18.75	100	22.92	97.92	72.80
	gpt-5-08-07	18.75	100	20.83	100	65.00
	gpt-5-mini-08-07	10.42	97.92	10.42	100	59.80
	gpt-5-nano-08-07	4.17	85.42	4.17	100	34.80
	o3-2025-04-16	4.17	93.75	6.25	100	58.40
	gpt-4.1-2025-04-14	2.08	87.50	10.42	97.92	39.58
	gpt-4o-2024-11-20	6.25	97.92	6.25	100	21.62
	gpt-oss-120b	2.08	100	6.25	100	26.00
DeepSeek	deepseek-v3p2	20.83	95.83	23.40	95.83	70.00
	deepseek-v3p1	16.67	97.92	10.42	100	—
	deepseek-r1-0528	10.42	100	8.33	100	57.60
Zhipu AI	glm-5	8.33	97.92	37.50	100	72.80
	glm-4p7	4.17	100	39.58	97.92	—
	glm-4p5	16.67	97.92	16.67	100	54.20
Qwen	qwen3-coder-480b-a35b	14.58	97.92	14.58	97.92	55.40
Moonshot AI	kimi-k2p5	22.92	97.92	25.00	97.92	70.80
	kimi-k2-instruct	16.67	100	18.75	100	43.80

Table 2 | Results on SWE-EVO with *release note + PR/issue context*: Resolved and Apply rates for OpenHands and SWE-agent. †Results reported on swebench.com (Jimenez et al., 2024). "—" indicates that the result is not reported on the SWE-Bench website.

The scale differences in Figure 4 translate into a large performance gap. Even the strongest model `gpt-5.4` resolves only around **25%** of SWE-EVO instances, and matched-model comparisons show the same pattern: `gpt-5.2` drops from 72.80% on SWE-bench Verified to 22.92% on SWE-



(a) GPT-series models.

(b) Open-source models.

Figure 5 | Failure mode distribution for SWE-agent with several model trajectories of unresolved instances. Each instance is automatically labeled using `gpt-5-mini` (OpenAI, 2025b) with categories from Table 8.

EVO. Performance still follows intuitive scaling trends across model families. One exception is `glm-5/glm-4p7`, which perform much better with SWE-agent than OpenHands, suggesting scaffold sensitivity in addition to benchmark difficulty. Taken together, these results indicate that SWE-EVO is a more demanding and more realistic benchmark for end-to-end software-evolution capabilities.

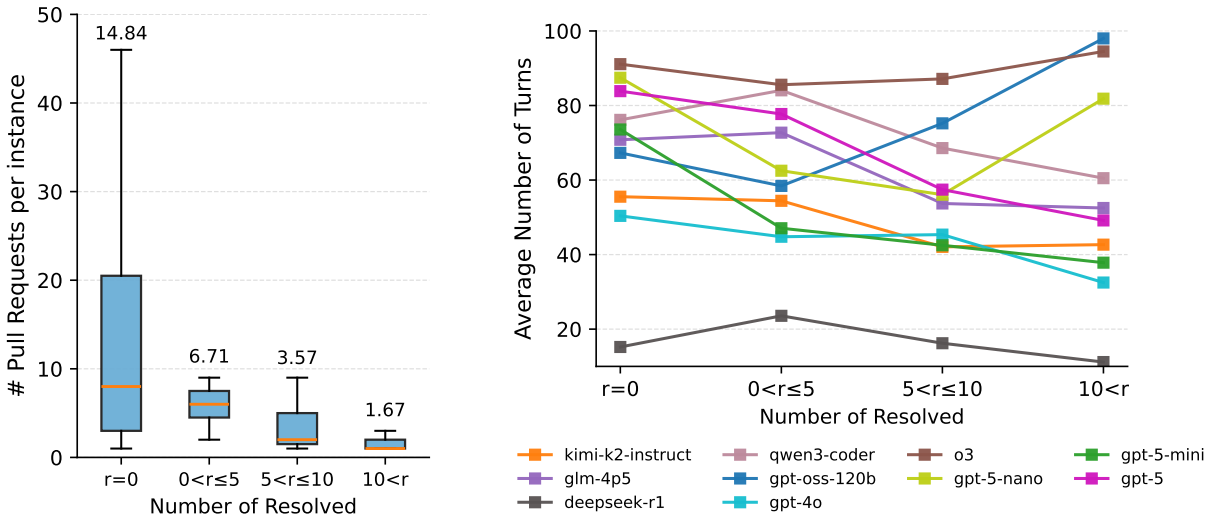
We also evaluate a more challenging *release-note only* setting, where agents receive no PR/issue context. As shown in Table 4 (in Appendix E), performance drops modestly but trends remain consistent, confirming that SWE-EVO poses substantial challenges regardless of specification detail.

Uncertainty from Benchmark Scale. Because SWE-EVO contains 48 curated instances, one resolved instance changes Resolved Rate by 2.08 percentage points. We therefore report uncertainty and avoid over-interpreting close leaderboard gaps. At the same time, the larger behavioral deltas in Figure 4 make each instance closer to a release-sized evolution task than to a single-issue repair task. Representative 95% Wilson confidence intervals over task instances are [14.9, 38.8] for 25.00%, [10.2, 31.9] for 18.75%, [4.5, 22.2] for 10.42%, and [0.4, 10.9] for 2.08%, so we focus on large performance gaps rather than small ranking differences.

Fine-Grained Analysis with Fix Rate. Beyond the binary *Resolved Rate*, we examine our soft metric *Fix Rate* (Section 3.2), which captures partial progress on large test suites. As shown in Table 6 (Appendix I), models indistinguishable under Resolved Rate can differ meaningfully: e.g., under OpenHands, both `gpt-4.1` and `gpt-oss-120b` resolve only 2.08% of SWE-EVO, but their Fix Rates are 4.65% vs. 2.08%, revealing that `gpt-4.1` repairs more failing tests per instance.

4.3. Analysis of Agent Behaviour

Beyond aggregate metrics, we investigate *why* agents fail on SWE-EVO by analyzing their execution trajectories. This qualitative analysis reveals distinct failure patterns across different model families.



(a) Average number of pull requests per difficulty group.

(b) Average number of turns by model across difficulty groups.

Figure 6 | Difficulty analysis of SWE-EVO instances: (a) instances associated with more pull requests are harder; (b) stronger models scale turn usage with difficulty, while others show limited adaptivity.

4.3.1. Trajectory Failure Modes Analysis

We perform an LLM-as-a-judge analysis of unresolved SWE-agent trajectories, following prior work on SWE-agent (Yang et al., 2024a). The judge assigns one primary label from the taxonomy in Table 8; these labels are qualitative diagnostics rather than execution-based scores, and human agreement validation remains future work. The taxonomy separates low-level execution failures from semantic failures: *Syntax Error* and *Tool-Use* capture invalid patches or failed interaction with the agent tools; *Incorrect Implementation* and *Instruction Following* capture wrong logic or misread requirements; *Stuck in Loop* and *Gave Up Prematurely* capture unproductive or prematurely terminated trajectories; and *Other* covers rare ambiguous cases.

Results. Figure 5 suggests distinct failure patterns across model families. For the GPT series, `gpt-5` rarely fails due to syntax or tool issues; instead, over 60% of failures come from *Instruction Following*, suggesting difficulty interpreting complex release notes. Smaller variants (`gpt-5-mini`, `gpt-5-nano`) show increasing *Incorrect Implementation*, *Tool-Use*, and *Syntax Error* failures, while older models (`o3`, `gpt-4.1`, `gpt-4o`) exhibit more looping and early-termination issues. In contrast, open and hybrid models follow different patterns: `kimi-k2-instruct`, `qwen3-coder`, and `gpt-oss-120b` fail mainly due to *Incorrect Implementation*, indicating weaker semantic reasoning but stable tool use, whereas `deepseek-r1` often gets stuck in loops or fails in execution, and `glm-4p5` shows more evenly distributed errors across categories.

4.3.2. Difficulty Analysis

A key property of SWE-EVO is that instance difficulty varies widely across multiple axes: PR count, specification length, files and lines edited, functions edited, and test-suite size (Table 1). Figure 6 groups instances by empirical difficulty, using the number of model-scaffold combinations that resolve each instance: $r = 0$, $0 < r \leq 5$, $5 < r \leq 10$, and $r > 10$, covering roughly 64%, 15%, 15%, and 6% of instances. Panel (a) shows that harder groups have substantially more linked PRs: unresolved-by-all

instances average 14.84 PRs, compared with 1.67 for the easiest group. This supports PR count as a useful proxy for release-level complexity. Panel (b) shows that stronger models tend to spend more turns on harder instances and terminate earlier on easier ones, while weaker models show less adaptive turn allocation.

5. Conclusion

We introduce SWE-EVO, a benchmark for evaluating coding agents on realistic software evolution rather than isolated bug fixing. SWE-EVO requires interpreting release-note-centered specifications, performing multi-file changes, and preserving functionality under comprehensive tests, making it substantially more challenging than SWE-Bench. Experiments with OpenHands and SWE-agent across 18 models reveal a large capability gap: the best model (`gpt-5.4`) solves only 25% of tasks, while a matched model such as `gpt-5.2` drops from 72.80% on SWE-Bench Verified to 22.92% on SWE-EVO. An exploratory failure analysis suggests that stronger models struggle more with instruction following, while weaker ones exhibit tool-use and syntax errors. We expect SWE-EVO to complement existing benchmarks with a focus on long-horizon software evolution.

6. Limitations

Our work has some limitations. First, SWE-EVO currently covers Python library projects only; this choice keeps the benchmark reproducible and compatible with SWE-Bench/SWE-Gym environments, but multilingual and downstream-application evolution remain future work. Second, our task specifications are release-note-centered, optionally augmented with linked PR/issue text, and therefore do not cover all evolution drivers, such as security advisories, dependency updates, or design-document-driven refactors. Third, the benchmark has 48 curated instances and an imbalanced repository distribution, especially toward `dvc`. We deliberately prioritize large, execution-validated release transitions over a larger number of shallow tasks, but the current size still limits statistical power for fine-grained comparisons; we therefore provide repository-composition details in Appendix H. Finally, Fix Rate is execution-verifiable but still weights tests equally and does not measure code quality, maintainability, or patch minimality. We plan to release task metadata, construction scripts, Docker execution settings, generated patches, and aggregate result files to support reproducibility.

References

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection. *International Conference on Learning Representations*, 2024.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. Swe-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. *arXiv preprint arXiv:2505.20411*, 2025.

Nghi DQ Bui, Hung Le, Yue Wang, Junnan Li, Akhilesh Deepak Gotmare, and Steven CH Hoi. Codetf: One-stop transformer library for state-of-the-art code llm. *arXiv preprint arXiv:2306.00029*, 2023.

Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. In *International Conference on Representation Learning*, 2024.

Quentin Carbonneaux, Gal Cohen, Jonas Gehring, Jacob Kahn, Jannik Kossen, Felix Kreuk, Emily McMilin, Michel Meyer, Yuxiang Wei, David Zhang, et al. Cwm: An open-weights llm for research on code generation with world models. *arXiv preprint arXiv:2510.02387*, 2025.

Aili Chen, Aonian Li, Bangwei Gong, Binyang Jiang, Bo Fei, Bo Yang, Boji Shan, Changqing Yu, Chao Wang, Cheng Zhu, et al. Minimax-m1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025.

Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. Codet: Code generation with generated tests. *arXiv preprint arXiv:2207.10397*, 2022.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021a.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021b.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*, 2023.

Cognition. Devin ai, 2024. https://cognition.ai/blog/introducing-devin.

DeepSeek-AI. Deepseek-r1-0528 release. https://api-docs.deepseek.com/news/news250528, 2025a. Accessed 2026-03-12.

DeepSeek-AI. Deepseek-v3.1 release. https://api-docs.deepseek.com/news/news250821, 2025b. Accessed 2026-03-12.

DeepSeek-AI. Deepseek-v3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025c.

DeepSeek AI. DeepSeek V3.1, 2025. https://api-docs.deepseek.com/news/news250821.

DORA Research Program. State of ai-assisted software development: 2025 dora report. https://cloud.google.com/resources/content/2025-dora-ai-assisted-software-development-report, September 2025. DevOps Research and Assessment (DORA).

Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, pages 31–53. IEEE, 2023.

Cuiyun Gao, Xing Hu, Shan Gao, Xin Xia, and Zhi Jin. The current challenges of software engineering in the era of large language models. *ACM Transactions on Software Engineering and Methodology*, 34(5):1–30, 2025.

Tao Ge, Jing Hu, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. *International Conference on Learning Representations*, 2024.

GLM-4.5 Team. Glm-4.5: Agentic, reasoning, and coding (arc) foundation models. *arXiv preprint arXiv:2508.06471*, 2025.

Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models. *Advances in Neural Information Processing Systems*, 37, 2024.

Junda He, Christoph Treude, and David Lo. Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead. *ACM Transactions on Software Engineering and Methodology*, 34(5):1–30, 2025.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*, 2021.

Anh Nguyen Hoang, Minh Le-Anh, Bach Le, and Nghi DQ Bui. Codewiki: Evaluating ai's ability to generate holistic documentation for large-scale codebases. *arXiv preprint arXiv:2510.24428*, 2025.

Qishuo Hua, Lyumanshan Ye, Dayuan Fu, Yang Xiao, Xiaojie Cai, Yunze Wu, Jifan Lin, Junfei Wang, and Pengfei Liu. Context engineering 2.0: The context of context engineering. *arXiv preprint arXiv:2510.26493*, 2025.

Dong Huang, Jie M Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. Agentcoder: Multi-agent-based code generation with iterative testing and optimisation. *arXiv preprint arXiv:2312.13010*, 2023.

Kush Jain, Gabriel Synnaeve, and Baptiste Roziere. Testgeneval: A real world unit test generation and test completion benchmark. *arXiv preprint arXiv:2410.00752*, 2024.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

Uttamjit Kaur and Gagandeep Singh. A review on software maintenance issues and how to reduce maintenance efforts. *International Journal of Computer Applications*, 118(1):6–11, 2015.

Kimi Team. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025a.

Kimi Team. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025b.

Kimi Team. Kimi k2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.

Jiayi Kuang, Yinghui Li, Xin Zhang, Yangning Li, Di Yin, Xing Sun, Ying Shen, and Philip S. Yu. Process-level trajectory evaluation for environment configuration in software engineering agents. *arXiv preprint arXiv:2510.25694*, 2025.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.

Michael Luo, Naman Jain, Jaskirat Singh, Sijun Tan, Ameen Patel, Qingyang Wu, Alpay Ariyak, Colin Cai, Shang Zhu Tarun Venkat, Ben Athiwaratkun, et al. Deepswe: Training a fully open-sourced, state-of-the-art coding agent by scaling rl, 2025.

Dung Nguyen Manh, Nam Le Hai, Anh TV Dau, Anh Minh Nguyen, Khanh Nghiem, Jin Guo, and Nghi DQ Bui. The vault: A comprehensive multilingual dataset for advancing code understanding and generation. *arXiv preprint arXiv:2305.06156*, 2023.

Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, et al. A survey of context engineering for large language models. *arXiv preprint arXiv:2507.13334*, 2025.

Dung Manh Nguyen, Thang Chau Phan, Nam Le Hai, Tien-Thong Doan, Nam V Nguyen, Quang Pham, and Nghi DQ Bui. Codemmlu: A multi-task benchmark for assessing code understanding & reasoning capabilities of codellms. In *The Thirteenth International Conference on Learning Representations*, 2025a.

Minh Huynh Nguyen, Thang Phan Chau, Phong X Nguyen, and Nghi DQ Bui. Agilecoder: Dynamic collaborative agents for software development based on agile methodology. In *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*, pages 156–167. IEEE, 2025b.

OpenAI. Hello gpt-4o. https://openai.com/index/hello-gpt-4o/, 2024a. Accessed 2026-03-12.

OpenAI. Swe-bench verified, 2024b. https://openai.com/index/introducing-swe-bench-verified/.

OpenAI. Introducing gpt-4.1 in the api. https://openai.com/index/gpt-4-1/, 2025a. Accessed 2026-03-12.

OpenAI. Gpt-5 system card. Technical report, OpenAI, 2025b. URL https://cdn.openai.com/gpt-5-system-card.pdf.

OpenAI. Introducing gpt-5.2. https://openai.com/index/introducing-gpt-5-2/, 2025c. Accessed 2026-03-12.

OpenAI. Gpt-5 system card. https://openai.com/index/gpt-5-system-card/, 2025d. Accessed 2026-03-12.

OpenAI. gpt-oss-120b & gpt-oss-20b model card. https://cdn.openai.com/pdf/419b6906-9da6-406c-a19d-1bb078ac7637/oai_gpt-oss_model_card.pdf, 2025e. Accessed 2026-03-12.

OpenAI. Introducing openai o3 and o4-mini. https://openai.com/index/introducing-o3-and-o4-mini/, 2025f. Accessed 2026-03-12.

OpenAI. Introducing gpt-5.4. https://openai.com/index/introducing-gpt-5-4/, 2026. Accessed 2026-03-12.

Charles Packer et al. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym, 2024. URL https://arxiv.org/abs/2412.21139.

Minh VT Pham, Huy N Phan, Hoang N Phan, Cuong Le Chi, Tien N Nguyen, and Nghi DQ Bui. Swe-synth: Synthesizing verifiable bug-fix data to enable large language models in resolving real-world bugs. *arXiv preprint arXiv:2504.14757*, 2025.

Huy Nhat Phan, Phong X. Nguyen, and Nghi D. Q. Bui. Hyperagent: Generalist software engineering agents to solve coding tasks at scale. *CoRR*, abs/2409.16299, 2024. doi: 10.48550/ARXIV.2409.16299. URL https://doi.org/10.48550/arXiv.2409.16299.

Cheng Qian, Chi Han, Yi R. Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. Creator: Tool creation for disentangling abstract and concrete reasoning of large language models, 2024.

Jiahao Qiu, Xuan Qi, Tongcheng Zhang, Xinzhe Juan, Jiacheng Guo, Yifu Lu, Yimin Wang, Zixin Yao, Qihan Ren, Xun Jiang, Xing Zhou, Dongrui Liu, Ling Yang, Yue Wu, Kaixuan Huang, Shilong Liu, Hongru Wang, and Mengdi Wang. Alita: Generalist agent enabling scalable agentic reasoning with minimal predefinition and maximal self-evolution, 2025.

Qwen Team. Qwen3-coder. https://github.com/QwenLM/Qwen3-Coder, 2025. Accessed 2026-03-12.

Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025.

Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. Raptor: Recursive abstractive processing for tree-organized retrieval. *International Conference on Learning Representations*, 2024.

Scale AI. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.

Chamkaur Singh, Neeraj Sharma, and Narender Kumar. Analysis of software maintenance cost affecting factors and estimation models. *Int. J. Sci. Technol. Res*, 8(9):276–281, 2019.

Hung Quoc To, Minh Huynh Nguyen, and Nghi DQ Bui. Functional overlap reranking for neural code generation. *arXiv preprint arXiv:2311.03366*, 2023.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024a.

Wenhan Wang, Chenyuan Yang, Zhijie Wang, Yuheng Huang, Zhaoyang Chu, Da Song, Lingming Zhang, An Ran Chen, and Lei Ma. Testeval: Benchmarking large language models for test case generation. *arXiv preprint arXiv:2406.04531*, 2024b.

Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley-gödel machine: Human-level coding agent development by an approximation of the optimal self-improving machine, 2025.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. In *Forty-first International Conference on Machine Learning*, 2024c.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024d.

Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi DQ Bui, Junnan Li, and Steven CH Hoi. Codet5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv:2305.07922*, 2023.

Zhiruo Wang, Daniel Fried, and Graham Neubig. Trove: Inducing verifiable and efficient toolboxes for solving programmatic tasks, 2024e.

Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. Magicoder: Source code is all you need. *arXiv preprint arXiv:2312.02120*, 2023.

Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. Swe-rl: Advancing llm reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*, 2025.

Chunqiu Steven Xia and Lingming Zhang. Conversational automated program repair. *arXiv preprint arXiv:2301.13246*, 2023.

Chunqiu Steven Xia and Lingming Zhang. Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using chatgpt. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 819–831, 2024.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024a.

John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, et al. Swe-bench multimodal: Do ai systems generalize to visual software domains? *arXiv preprint arXiv:2410.03859*, 2024b.

Haoran Ye, Xuning He, Vincent Arak, Haonan Dong, and Guojie Song. Meta context engineering via agentic skill evolution. *arXiv preprint arXiv:2601.21557*, 2026.

Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. Evaluating and improving chatgpt for unit test generation. *Proceedings of the ACM on Software Engineering*, 1(FSE):1703–1726, 2024.

Z.ai. Glm-4.7: Advancing the coding capability. https://z.ai/blog/glm-4.7, 2025. Accessed 2026-03-12.

Z.ai Team. Glm-5: From vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.

Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, et al. Multi-swe-bench: A multilingual benchmark for issue resolving. *arXiv preprint arXiv:2504.02605*, 2025.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin godel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025a.

Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. Swe-bench goes live! *arXiv preprint arXiv:2505.23419*, 2025b.

Quanjun Zhang, Chunrong Fang, Yang Xie, Yaxin Zhang, Yun Yang, Weisong Sun, Shengcheng Yu, and Zhenyu Chen. A survey on large language models for software engineering. *arXiv preprint arXiv:2312.15223*, 2023.

Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. Autocoderover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 1592–1604, Vienna, Austria, 2024. ACM.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. *AAAI Conference on Artificial Intelligence*, 2024.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning acting and planning in language models. *arXiv preprint arXiv:2310.04406*, 2023.

Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.

A. Evaluated Models

We evaluate SWE-EVO on a diverse set of recent LLMs from multiple providers, including both closed- and open-weight models. Table 3 lists all evaluated models.

Provider	Model	Type	Reference
OpenAI	gpt-5.4	Closed-weight	(OpenAI, 2026)
	gpt-5.2	Closed-weight	(OpenAI, 2025c)
	gpt-5-08-07	Closed-weight	(OpenAI, 2025d)
	gpt-5-mini-08-07	Closed-weight	(OpenAI, 2025d)
	gpt-5-nano-08-07	Closed-weight	(OpenAI, 2025d)
	o3-2025-04-16	Closed-weight	(OpenAI, 2025f)
	gpt-4.1-2025-04-14	Closed-weight	(OpenAI, 2025a)
	gpt-4o-2024-11-20	Closed-weight	(OpenAI, 2024a)
	gpt-oss-120b	Open-weight	(OpenAI, 2025e)
DeepSeek	deepseek-v3p2	Open-weight	(DeepSeek-AI, 2025c)
	deepseek-v3p1	Open-weight	(DeepSeek-AI, 2025b)
	deepseek-r1-0528	Open-weight	(DeepSeek-AI, 2025a)
Zhipu AI	glm-5	Open-weight	(Z.ai Team, 2026)
	glm-4p7	Open-weight	(Z.ai, 2025)
	glm-4p5	Open-weight	(GLM-4.5 Team, 2025)
Qwen	qwen3-coder-480b-a35b	Open-weight	(Qwen Team, 2025; Yang et al., 2025)
Moonshot AI	kimi-k2p5	Open-weight	(Kimi Team, 2026)
	kimi-k2-instruct	Open-weight	(Kimi Team, 2025a)

Table 3 | Full list of LLMs evaluated on SWE-EVO, grouped by provider.

B. Software Evolution Conceptual Model

Software evolution is an iterative cycle where, starting from an existing system, engineers identify required changes, analyze their impact, and carry out an evolution process that yields a new system aligned with updated requirements. This cycle repeats as the new system becomes the starting point for subsequent iterations. SWE-EVO captures this process by tasking agents with evolving codebases between consecutive release versions.

C. Self-Evolving Agents and Tool Creation

Due to the huge design space for software agents, building an optimal agent scaffold can be extremely challenging and costly. As a result, several self-improving and self-evolving software agents have emerged recently. The Self-Improving Coding Agent (SICA) (Robeyns et al., 2025) introduced mechanisms for agents to learn from their own experiences. Darwin-Gödel Machine (DGM) (Zhang et al., 2025a) proposed open-ended evolution of self-improving agents through iterative refinement. Huxley-Gödel Machine (HGM) (Wang et al., 2025) further advanced this paradigm by approximating optimal self-improving machines for human-level coding. However, such self-improving agents typically require costly offline training on known benchmarks and may not generalize well across different LLMs, benchmarks, and issue types.

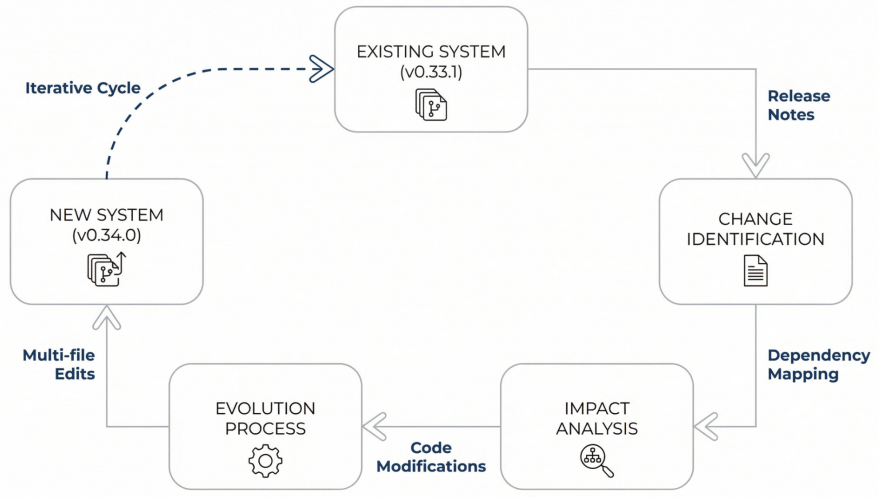


Figure 7 | Conceptual model of software evolution, depicting the iterative cycle from an existing system to a new system through change identification, impact analysis, and the evolution process.

Beyond the software engineering domain, prior work has explored using LLMs to create tools for general reasoning or embodied tasks. Large Language Models as Tool Makers (LATM) (Cai et al., 2024) demonstrated that LLMs can autonomously create reusable tools to solve complex tasks more efficiently. Voyager (Wang et al., 2024a) introduced an open-ended embodied agent that continuously acquires new skills through code generation in Minecraft. CREATOR (Qian et al., 2024) proposed disentangling abstract and concrete reasoning through tool creation. TroVE (Wang et al., 2024e) focused on inducing verifiable and efficient toolboxes for programmatic tasks. Alita (Qiu et al., 2025) presented a generalist agent enabling scalable agentic reasoning with minimal predefinition and maximal self-evolution. While these works demonstrate the potential of self-evolving agents and tool creation, they do not specifically target real-world software engineering problems that require long-horizon evolution across multiple commits and versions.

D. SWE-EVO vs. SWE-Bench Comparison

The main paper reports the quantitative comparison in Figure 4. The same statistics are derived from instance-level metadata for SWE-EVO and SWE-Bench, covering specification length, gold-patch scope, and test-suite size.

E. Context Comparison Results

We further compare performance under *release-note only* and *release-note + PR/issue context* settings. As shown in Table 4, adding PR/issue context generally improves resolved rates across most models, while preserving the overall ranking and relative trends. This suggests that additional context provides useful signals.

F. Pull Request Distribution Analysis

Unlike SWE-Bench, where each task instance corresponds to a single pull request (PR), an instance in SWE-EVO may be associated with multiple pull requests that collectively implement or refine a release-note change. We hypothesize that instances associated with a larger number of pull requests

	release-note only		release-note + PR/issue context	
Model	OpenHands	SWE-agent	OpenHands	SWE-agent
gpt-5-08-07	**14.58**	**16.67**	**18.75**	**20.83**
gpt-5-mini-08-07	8.33	10.42	10.42	10.42
o3-2025-04-16	4.17	6.25	4.17	6.25
gpt-4.1-2025-04-14	2.08	8.33	2.08	10.42
gpt-4o-2024-11-20	8.33	4.17	6.25	6.25
gpt-oss-120b	0.00	0.00	2.08	6.25
deepseek-r1-0528	10.42	6.25	10.42	8.33
glm-4p5	6.25	12.50	16.67	16.67
qwen3-coder-480b-a35b	**14.58**	12.50	14.58	14.58
kimi-k2-instruct	8.33	12.50	16.67	18.75

Table 4 | Results on SWE-EVO under two settings: **release-note only** and **release-note + PR/issue context**. Reported metric is **Resolved Rate (%)** on **OpenHands** and **SWE-agent**.

reflect more complex, multi-step development efforts, as each pull request typically represents a distinct feature enhancement or bug fix. As shown in Figure 8, the number of pull requests per instance varies widely, indicating that SWE-EVO spans a broad range of difficulty levels.

G. Difficulty Analysis

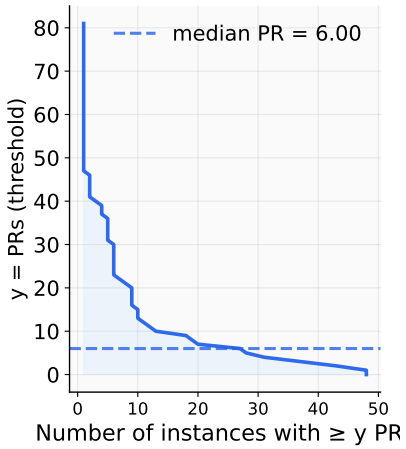
The main paper reports the difficulty analysis in Figure 6 and Section 4.3.2. We keep this appendix section as a pointer so the appendix PR-distribution analysis remains connected to the promoted main-text result without duplicating the same figure.

H. Repository Composition and Bias Checks

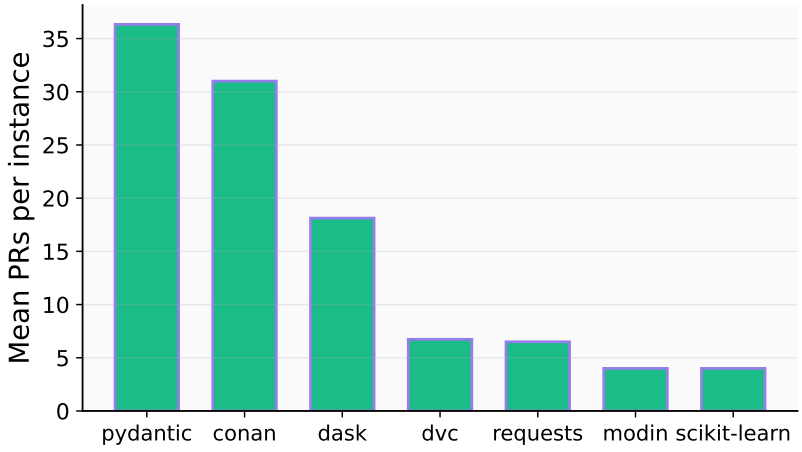
Table 5 reports per-repository Resolved Rate and Fix Rate, averaged over these evaluated models: `gpt-5-08-07`, `gpt-5-mini-08-07`, `gpt-5-nano-08-07`, `o3-2025-04-16`, `gpt-4.1`, `gpt-4o`, `gpt-oss-120b`, `deepseek-r1`, `glm-4p5`, `qwen3-coder-480b`, and `kimi-k2-instruct`. Our strict collection process requires stable release tags, reproducible environments, and executable tests, which improves validity but introduces some quantity imbalance: 26 of 48 instances come from `dvc`. However, the resulting metrics are not concentrated on a single repository. Performance remains low across repositories, and the highest aggregate values vary by scaffold. We therefore treat repository composition as a dataset caveat, but the metric itself does not appear to favor any single repository.

I. Fix Rate Results

Beyond the binary *Resolved Rate*, we also examine our soft metric *Fix Rate*, which captures partial progress on large test suites (Section 3.2). As shown in Table 6, models that appear similar under Resolved Rate can differ meaningfully once partial improvements are considered. For example, under OpenHands, both `gpt-4.1` and `gpt-oss-120b` resolve only 2.08% of SWE-EVO, but their Fix Rates differ (4.65% vs. 2.08%), indicating that `gpt-4.1` consistently fixes more failing tests per instance. More broadly, we observe a consistent gap between Fix Rate and Resolved Rate across all models, suggesting that many trajectories make partial but incomplete progress. This finer granularity is



(a) Complementary cumulative distribution of PR counts, showing how many instances contain at least y linked pull requests.



(b) Mean PRs per repository, highlighting that certain codebases require substantially more upstream context to resolve.

Figure 8 | Pull Request statistics across SWE-EVO instances.

Repository	# Inst.	Share (%)	OpenHands		SWE-agent	
			Resolved	Fix	Resolved	Fix
`dvc`	26	54.2	4.36	6.69	8.72	5.63
`dask`	8	16.7	2.68	1.14	2.57	0.83
`requests`	4	8.3	2.65	2.46	3.12	2.92
`pydantic`	3	6.2	0.00	0.00	0.01	0.00
`modin`	3	6.2	1.23	1.14	1.04	1.04
`conan`	2	4.2	0.38	0.38	0.31	0.21
`scikit-learn`	2	4.2	0.19	0.19	0.00	0.00
Total	48	100.0	–	–	–	–

Table 5 | Repository composition and aggregate per-repository performance. Resolved and Fix values are percentages averaged over these evaluated models for each scaffold.

especially important on SWE-EVO, where each task involves many tests, and binary outcomes would otherwise obscure systematic differences in model capability.

J. FAIL_TO_PASS and PASS_TO_PASS Diagnostics

Table 7 reports detailed `FAIL_TO_PASS` and `PASS_TO_PASS` rates for each model in these evaluated model and each scaffold. These diagnostics separate two effects that are combined by strict Fix Rate: whether the model implements the requested behavioral change, and whether it preserves tests that already passed before the patch. Across most models, higher `PASS_TO_PASS` rates tend to coincide with lower `FAIL_TO_PASS` rates, suggesting a trade-off: as a model spends more effort resolving the new task, it may be more likely to forget or break existing codebase behavior. However, valid software evolution should satisfy the requested change without breaking existing behavior. We therefore use these rates as diagnostics, while keeping strict Fix Rate as the main soft metric.

	Model	OpenHands		SWE-agent		Average	
		Resolved (%)	Fix (%)	Resolved (%)	Fix (%)	Resolved (%)	Fix (%)
OpenAI	gpt-5.4	**25.00**	**33.89**	**25.00**	**33.98**	**25.00**	**33.96**
	gpt-5.2	18.75	23.43	22.92	30.21	20.84	26.82
	gpt-5-08-07	18.75	27.64	20.83	31.46	19.79	29.55
	gpt-5-mini-08-07	10.42	17.48	10.42	17.48	10.42	17.48
	gpt-5-nano-08-07	4.17	5.99	4.17	5.26	4.17	5.63
	o3-2025-04-16	4.17	6.47	4.17	13.72	4.17	10.10
	gpt-4.1-2025-04-14	2.08	4.65	10.42	14.79	6.25	9.72
	gpt-4o-2024-11-20	6.25	7.77	6.25	10.15	6.25	8.96
	gpt-oss-120b	2.08	2.08	6.25	7.88	4.17	4.98
DeepSeek	deepseek-v3p2	20.83	26.89	23.40	31.41	22.12	29.15
	deepseek-v3p1	16.67	21.83	10.42	15.51	13.55	18.67
	deepseek-r1-0528	10.42	14.31	8.33	9.89	9.38	12.10
Zhipu AI	glm-5	8.33	9.67	37.50	44.27	22.92	27.45
	glm-4p7	4.17	5.19	39.58	45.87	21.88	25.53
	glm-4p5	16.67	23.74	16.67	26.55	16.67	25.15
Qwen	qwen3-coder-480b-a35b	14.58	19.56	14.58	23.74	14.58	21.65
Moonshot AI	kimi-k2p5	22.92	27.75	**25.00**	32.91	23.96	30.33
	kimi-k2-instruct	16.67	22.42	18.75	24.03	17.71	23.23

Table 6 | Comparison of **Resolved Rate (%)** and **Fix Rate (%)** across models on **OpenHands**, **SWE-agent**, and their **Average**.

K. Data and Code Availability

SWE-EVO is designed for public release. We will release the task metadata, release-note and PR/issue context construction scripts, Docker-based execution configuration, evaluation harness, generated patches, and aggregate result files needed to reproduce the reported metrics. The release will also include the dataset fields described in Table 9 and instructions for running OpenHands and SWE-agent under the settings used in this paper. When required for anonymous review, these artifacts should be distributed through an anonymized repository or supplemental archive.

L. Failure Mode Descriptions

We use a coarse taxonomy that separates execution failures, semantic failures, and unproductive trajectories. For each unresolved SWE-agent trajectory, the judge assigns exactly one primary label; when multiple issues are present, it chooses the most fundamental cause. Table 8 summarizes the labels used in the main analysis.

M. Dataset Fields

Table 9 describes the SWE-EVO dataset fields and outlines how they are obtained throughout the curation process.

Model	OpenHands		SWE-agent	
	FAIL_TO_PASS	PASS_TO_PASS	FAIL_TO_PASS	PASS_TO_PASS
gpt-5-08-07	65.68	34.32	59.95	37.97
gpt-5-mini-08-07	64.15	23.35	76.27	21.65
gpt-5-nano-08-07	58.59	8.07	90.58	7.34
o3-2025-04-16	85.20	8.55	82.11	15.81
gpt-4.1	82.85	4.65	71.67	24.16
gpt-4o	87.20	10.89	81.62	16.30
gpt-oss-120b	95.83	4.17	86.91	11.00
deepseek-r1	77.32	22.68	83.23	14.69
glm-4p5	68.64	29.28	62.57	35.35
qwen3-coder-480b	68.54	29.38	62.97	32.86
kimi-k2-instruct	71.45	28.55	67.52	30.39

Table 7 | FAIL_TO_PASS and PASS_TO_PASS rates (%) by model and scaffold. The diagnostics separate progress on required behavior from preservation of previously passing tests.

Category	Description
Syntax Error	Patch introduces parse, formatting, import, indentation, JSON/YAML, or similar errors that prevent execution.
Incorrect Implementation	Patch touches a plausible area but implements the required behavior incorrectly or incompletely.
Instruction Following	Agent misreads, ignores, or deviates from the release note or linked requirement, effectively solving the wrong task.
Tool-Use	Progress is blocked by failed or incorrect tool use, such as bad edits, missed tests, or wrong paths.
Stuck in Loop	Agent repeatedly reads, edits, or reruns tests without substantive progress.
Gave Up Prematurely	Agent stops or declares failure while reasonable next steps remain.
Other	Rare or ambiguous failure not captured by the other labels.

Table 8 | Descriptions of failure mode categories.

N. Problem Statement

For each SWE-EVO instance, the problem statement is defined by the official release notes text that describes the changes between the *start* version and the *end* version of this instance, as published by the project maintainers. This release note is then presented to the agent as the sole natural-language specification of how the codebase should change between the two versions.

N.1. Pull Request, Issue Context

Release notes frequently refer to specific pull requests or issues (e.g., “see #1234” or “thanks to PR #5678”). To make these references actionable for agent, we augment the problem statement with the context of the linked artifacts. For each referenced pull request or issue, we fetch its body from github. These texts are then concatenated into a compact “PR / Issue Context” block that is presented to the agent alongside the release note. We then append these texts directly below the release note in a structured format, without rewriting or summarizing them. In general, the final problem statement presented to the agent has the form: *release-note* followed by a sequence of sections

Field	Type	Description
repo	str	Git repository identifier for the task instance, e.g., the GitHub owner/name.
base_commit	str	The commit on which the pull request is based, representing the repository state before the issue is resolved.
start_version	str	Git tag or version identifier of the starting release for this task instance.
end_version	str	Git tag or version identifier of the target release after the change has been applied.
end_version_commit	str	Git commit hash corresponding to the end_version tag.
patch	str	Gold patch proposed by the pull request, in .diff format.
test_patch	str	Modifications to the test suite proposed by the pull request that are typically used to check whether the issue has been resolved.
problem_statement	str	Issue description text, typically describing the bug or requested feature, used as the task problem statement.
FAIL_TO_PASS	List[str]	Test cases that are expected to successfully transition from failing to passing and are used to evaluate the correctness of the patch.
PASS_TO_PASS	List[str]	Test cases that are already passing prior to applying the gold patch; a correct patch should not introduce regression failures in these tests.
*image	str	Instance-level Docker image that provides an execution environment.
*test_cmds	List[str]	Command(s) used to run the test suite, as identified by the verify agent in RepoLaunch, enabling detailed logging of each test item's status (e.g., via pytest -rA).
*log_parser	str	Type of log parser required for the instance—by default, pytest.

Table 9 | Required fields for a typical issue-solving task instance. Fields marked with * are newly added compared to SWE-Bench.

\n### PR xxx:\n or \n### Issue yyy:\n with their corresponding content for every referenced pull request and issue. so that the agent sees the original release note followed by the full text of any referenced pull requests and issues.

N.2. Example

To make this concrete, we present two example problem statements (Boxes N.2 and N.2), corresponding to the instances iterative__dvc_0.33.1_0.34.0 in the iterative/dvc repository and dask__dask_2023.6.1_2023.7.0 in the dask/dask repository.

Problem Statement: iterative__dvc_0.33.1_0.34.0

1) [‘dvc metrics show‘ now nicely formats multiline metrics files like tsv/htsv, csv/hcsv, json](https://github.com/iterative/dvc/issues/1716); Kudos @mroutis :medal_sports:
2) [‘dvc remote add‘ no longer silently overwrites existing sections](https://github.com/iterative/dvc/issues/1760);
3) [Use a workaround to bypass SIP protection on osx, when accessing libSystem](https://github.com/iterative/dvc/issues/1515);
4) [Don’t try to create existing container on azure](https://github.com/iterative/dvc/issues/1811); Kudos @AmitAronovitch :medal_sports:
5) Dvc repository now uses read-only http remote for our images instead of s3;
6) Fix bug in ‘dvc status‘ where an error is raised if cache is not present locally nor on the remote; 7) [Fix progress bar on ‘dvc pull‘](https://github.com/iterative/dvc/issues/1807); Kudos @pared :medal_sports:
8) [Automatically detect metrics file type by extension](https://github.com/iterative/dvc/issues/1553); Kudos @cand126 :medal_sports:

Welcome new contributor @AmitAronovitch ! :tada:

Issue 1716:
When displaying TSV metrics output the printout is not readable:
```
../../20181223-TrainSetZero/SanityCheck/Bravo_on_TrainSetZero.metrics:  value_mse
deviation_mse data_set
0.421601 0.173461 train
0.67528 0.289545 testing
0.671502 0.297848 validation
```
I think it would be much easier to read if a newline were added, and the remaining text were formatted as though it had been passed via `column -t`:
```
../../20181223-TrainSetZero/SanityCheck/Bravo_on_TrainSetZero.metrics:
value_mse deviation_mse data_set
0.421601 0.173461 train
0.67528 0.289545 testing
0.671502 0.297848 validation
```

Issue 1807:
```
(3.7.0-dvc) →dvc git:(dvctags) ×dvc pull
Preparing to download data from 's3://dvc-share/dvc/'
Preparing to collect status from s3://dvc-share/dvc/
[##############################] 100% Collecting information
[##############################] 100% Analysing status.
(1/3):  [##############################] 100% dvc_up.bmp
(2/3):  [##############################] 100% dvc.ico
(3/3):  [##############################] 100% dvc_left.bmp
(4/3):  [##############################] 100% Checkout finished!
```
looks like checkout didn’t reset progress counter

Issue 1553:
E.g. if we see that output has .json suffix, we could safely assume that it is `-type json` without explicitly specifying it.

Problem Statement: iterative__dvc_0.33.1_0.34.0 (continued)

Issue 1760:
The `dvc remote add` command ignores the existing remote and overwrites it silently.
```
→dvc -version
0.32.1+7d7ed4
```
To reproduce
```
dvc remote add s3 s3://bucket/subdir
dvc remote add s3 s3://bucket/subdir2
```
Expected behavior
The second command `dvc remote add s3 s3://bucket/subdir2` should fail with the Remote with name "s3" already exists message.
Current behavior
Remote URL is silently overwriten:
```
> cat .dvc/config
[ 'remote "s3"' ]
url = s3://bucket/subdir2
```

Issue 1811:
Azure SAS connection strings can be used to allow read-only access to specific container, which is useful in the context of CI pipelines and automation.
However, when using such limited-credentials connection string, `dvc pull` abends with the error below.
Some digging reveals that this is because `remote/azure.py` always tries to create the bucket (which already exists). If we check for existence before trying to create - this should work...
(will send PR)
```
[######### ] 30% Collecting information
Client-Request-ID=19129d6a-5393-11e9-80ab-5800e34ea0d9
Retry policy did not allow for a retry:
Server-Timestamp=Sun, 31 Mar 2019 08:58:06 GMT,
Server-Request-ID=c72fbeda-501e-0089-3c9f-e78298000000,
HTTP status code=403, Exception=This request is not
authorized to perform this operation.
ErrorCode:  AuthorizationFailure
<?xml version="1.0" encoding="utf-8"?><Error><Code>AuthorizationFailure</Code>
<Message>This request is not authorized to perform this operation.
RequestId:c72fbeda-501e-0089-3c9f-e78298000000
Time:2019-03-31T08:58:06.2059267Z</Message></Error>.
```
Error: failed to pull data from the cloud - This request is not authorized to perform this operation. ErrorCode: AuthorizationFailure
```
<?xml version="1.0" encoding="utf-8"?><Error><Code>AuthorizationFailure
</Code><Message>This request is not authorized to perform this operation.
RequestId:c72fbeda-501e-0089-3c9f-e78298000000
Time:2019-03-31T08:58:06.2059267Z</Message></Error>
```
Having any troubles? Hit us up at https://dvc.org/support, we are always happy to help!
Please provide information about your setup
DVC version: 0.29.0+220b4b.mod
linux x86_64 py3.6 pip

Issue 1515:
```
$ dvc -V
0.23.2+bad2ef.mod
```
DVC creates hardlinks instead of reflinks in APFS.
File system:
```
$ diskutil info / | grep -i system
File System Personality:  APFS
```

Problem Statement: iterative__dvc_0.33.1_0.34.0 (continued)

See last two lines:

```
$ dvc add Tags.xml -v
Debug:  PRAGMA user_version;
Debug:  fetched:  [(3,)]
Debug:  CREATE TABLE IF NOT EXISTS state (inode INTEGER PRIMARY KEY, mtime TEXT NOT NULL, size
TEXT NOT NULL, md5 TEXT NOT NULL, timestamp TEXT NOT NULL)
Debug:  CREATE TABLE IF NOT EXISTS state_info (count INTEGER)
Debug:  CREATE TABLE IF NOT EXISTS link_state (path TEXT PRIMARY KEY, inode INTEGER NOT NULL,
mtime TEXT NOT NULL)
Debug:  INSERT OR IGNORE INTO state_info (count) SELECT 0 WHERE NOT EXISTS (SELECT * FROM
state_info)
Debug:  PRAGMA user_version = 3;
Debug:  Skipping copying for '/Users/dmitry/src/modules-example/tmp/
test/Tags.xml', since it is not a symlink or a hardlink.
Adding 'Tags.xml' to '.gitignore'.
Saving 'Tags.xml' to cache '.dvc/cache'.
Debug:  Path /Users/dmitry/src/modules-example/tmp/test/Tags.xml inode 12888021464
Debug:  SELECT * from state WHERE inode=12888021464
Debug:  fetched:  []
Debug:  INSERT INTO state(inode, mtime, size, md5, timestamp) VALUES (12888021464,
"1547854167848187904", "0", "6d861a1605e60b2f3a977a5a2a4419a2", "1547854178609126912")
Debug:  File '/Users/dmitry/src/modules-example/tmp/test/.dvc/cache/6d/
861a1605e60b2f3a977a5a2a4419a2', md5 '6d861a1605e60b2f3a977a5a2a4419a2', actual 'None'
Debug:  Cache type 'reflink' is not supported:  reflink is not supported
Debug:  Created 'hardlink':  /Users/dmitry/src/modules-example/tmp/
test/.dvc/cache/6d/861a1605e60b2f3a977a5a2a4419a2 →/Users/dmitry/src/
modules-example/tmp/test/Tags.xml
```

Problem Statement: dask__dask_2023.6.1_2023.7.0

2023.7.0
Released on July 7, 2023
Enhancements

- Catch exceptions when attempting to load CLI entry points (#10380) Jacob Tomlinson

Bug Fixes

- Fix typo in `_clean_ipython_traceback` (#10385) Alexander Clausen
- Ensure that `df` is immutable after `from_pandas` (#10383) Patrick Hoefler
- Warn consistently for `inplace` in `Series.rename` (#10313) Patrick Hoefler

Documentation

- Add clarification about output shape and reshaping in rechunk documentation (#10377) Swayam Patil

Maintenance

- Simplify `astype` implementation (#10393) Patrick Hoefler
- Fix `test_first_and_last` to accommodate deprecated `last` (#10373) James Bourbeau
- Add `level` to `create_merge_tree` (#10391) Patrick Hoefler
- Do not derive from `scipy.stats.chisquare` docstring (#10382) Doug Davis

PR 10385:
Regression from (#10354) https://api.github.com/repos/dask/dask/pull/10354 - exception types of unhandled exceptions in ipython contexts were mangled and always set to type.
To reproduce, run the following in a jupyter notebook cell with dask 2023.6.1:

```
import dask
raise TypeError("wat")
```

Then, observe the websocket connection to jupyter; the message with `msg_type="error"` looks like this:
(Image Link)
Notice `content["ename"]` is set to "type".
With dask 2023.6.0, `content["ename"]` is properly set to "TypeError":
(Image Link)
These values aren't inconsequential - they end up being serialized into the ipynb file, and cause hard to debug issues.
(nb: I would prefer if this kind of issue was not something that can be caused by `import dask`, by opting in to this functionality by configuration or actively registering the hook functions)

- Tests added / passed
- Passes `pre-commit run --all-files`

Thanks to (@matbryan52) for the help debugging this.

Problem Statement: dask__dask_2023.6.1_2023.7.0 (continued)

PR 10383:

- Tests added / passed
- Passes `pre-commit run --all-files`

We had a similar problem in dask-expr

PR 10393:

- Tests added / passed
- Passes `pre-commit run --all-files`

PR 10391:

- Passes `pre-commit run --all-files`

PR 10373:
This fixes
```
FAILED dask/dataframe/tests/test_dataframe.py::
test_first_and_last[last] - FutureWarning:  last is deprecated
and will be removed in a future version.  Please create a mask
and filter using .loc instead
```
which we're currently seeing in the upstream build (xref (#10347) (Workflow Run URL)
Python 3.10 Test Summary
```
dask/dataframe/tests/
test_multi.py::test_concat_categorical[True-False-True]:
[XPASS(strict)] fails on pandas dev:
https://api.github.com/repos/dask/dask/issues/10558
dask/dataframe/tests/
test_multi.py::test_concat_categorical[False-False-True]:
[XPASS(strict)] fails on pandas dev:
https://api.github.com/repos/dask/dask/issues/10558
```
Note that we were already emitting a FutureWarning for `DataFrame.last`, so this is a tests-only PR
cc (@j-bennet) (@phofl)

PR 10382:

- Closes (#10381) Looks like our docs are running into an error on main. See this build from a recent PR https://readthedocs.org/projects/dask/builds/21146384/
- x Tests added / passed
- Passes `pre-commit run --all-files`

`scipy.stats.chisquare` recently started using a doi Sphinx directive that exists in their repo but it's not importable. This is breaking dask docs generation. We can just point folks to that docstring instead of deriving from it.
for reference here is the PR in scipy: (scipy/scipy#17682)

PR 10377:

Closes (#10361) It is not documented anywhere that `dask.array.rechunk` expects the output to have the same shape as the input and does not allow reshaping. The validation step is also quite hidden. This has led to some confusion (dask/distributed#7897 (comment)), so I think it would be good to add a remark about this in the documentation.

- Tests added / passed
- Passes `pre-commit run --all-files`

PR 10313:
The previous warning seemed very inconsistent

PR 10380:

- Closes (#10379) If a package registers an entry point for the CLI but the entrypoint is broken or not importable the CLI cannot work at all.

This is a little hard to create an MRE for but I seeing this with dask-ctl when updating textual to a more recent version. The error comes from the following line.
(dask/dask/cli.py)
Line 97 in `/dask/dask/commit/85c99bc20abc382774cfb6e5bf5f2db76ac09378`
```
command = entry_point.load()
```
When this line is called third-party code is loaded, so we have no control over what happens. We could broadly catch exceptions here, print a warning and carry on.

- Tests added / passed
- Passes `pre-commit run --all-files`